Data Communications

Lecture #5

What we have discussed

- E-mail, SMTP, IMAP
- Domain Name System (DNS)

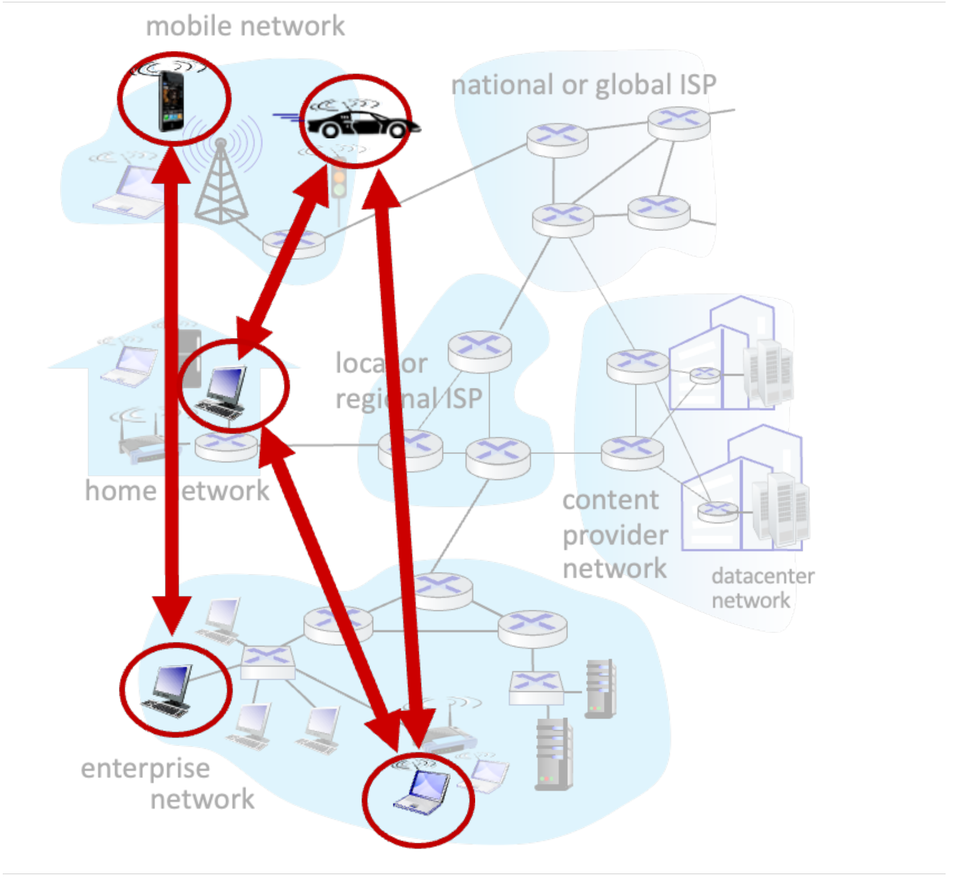
Today…

- P2P applications - bitTorrent, DHT

P2P Applications

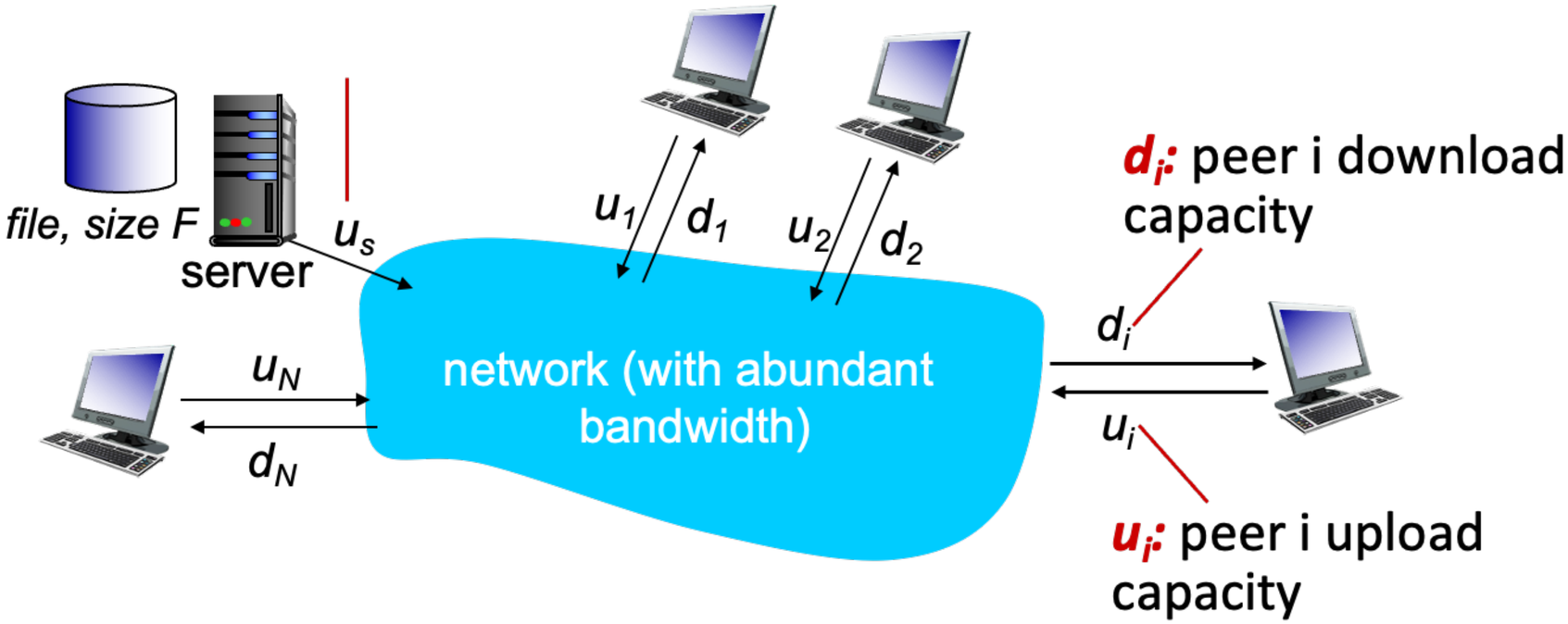
Peer-to-peer architecture

- No always-on server
- Arbitrary end systems directly communicate
- Peers request service from other peers, provide service in return to other peers
  - Self-scalability - new peers bring new service capacity, as well as new service demands
- Peers are intermittently connected and change IP addresses
  - Complex management
- e.g., P2P file sharing (bitTorrent), VoIP (Skype)



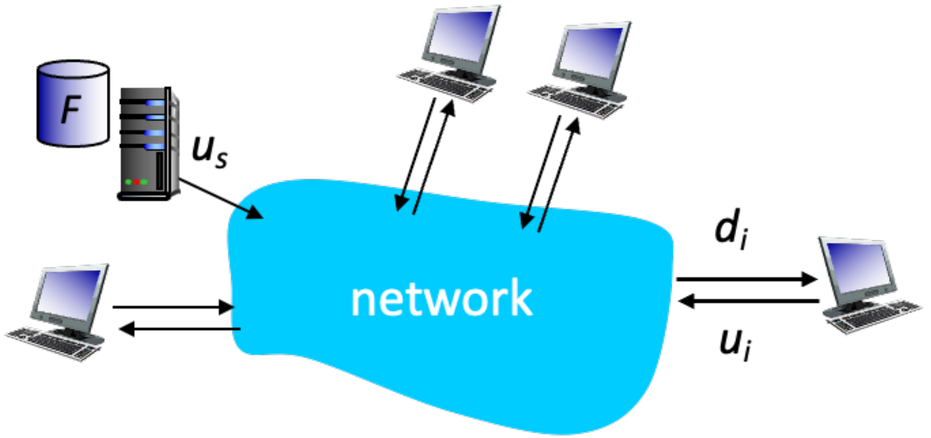
File distribution: client-server vs P2P

- How much time to distribute file (size F) from one server to N peers?
  - Peer upload/download capacity is limited resource



File distribution time: client-server

- Server transmission must sequentially send (upload) N file copies
  - Time to send one copy: $\frac{F}{u_s}$
  - Time to send N copies: $\frac{NF}{u_s}$



- Each client must download file copy
  - d_{min} = min client download rate
  - min client download time = $\frac{F}{d_{min}}$

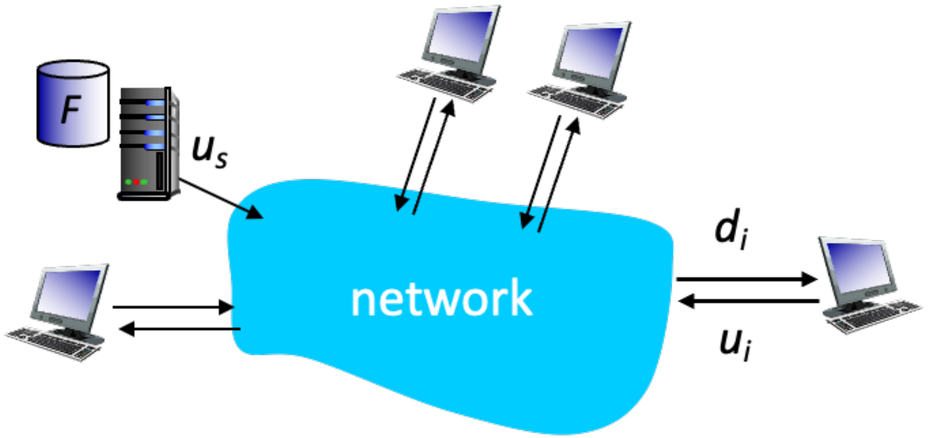
Time to distribute F to N clients using client-server approach

$$\Rightarrow D_{c-s} \geq \max\{\frac{NF}{u_s}, \frac{F}{d_{min}}\}$$

Increase linearly in N !

File distribution time: P2P

- Server transmission must upload at least one copy
  - Time to send one copy: $\frac{F}{u_s}$
- Each client must download file copy
  - min client download time = $\frac{F}{d_{min}}$
- As aggregate must download NF bits
  - max upload rate is (limiting max download rate) $= u_s + \sum u_i$



Time to distribute F to N clients using P2P approach

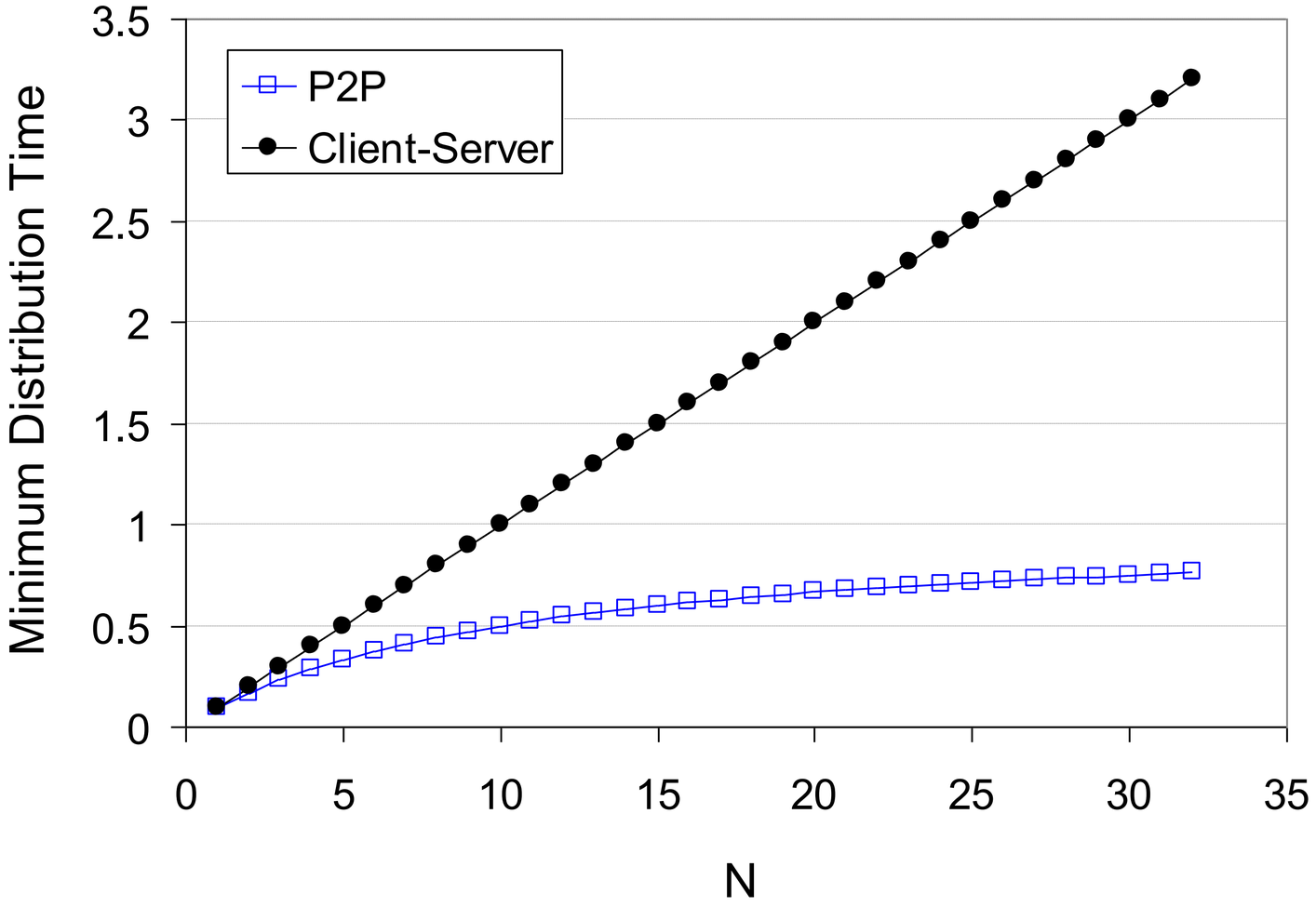
$\Rightarrow$

$$D_{c-s} \geq \max\{\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum u_i}\}$$

Increase linearly in N… but so does this, as each peer brings service capacity

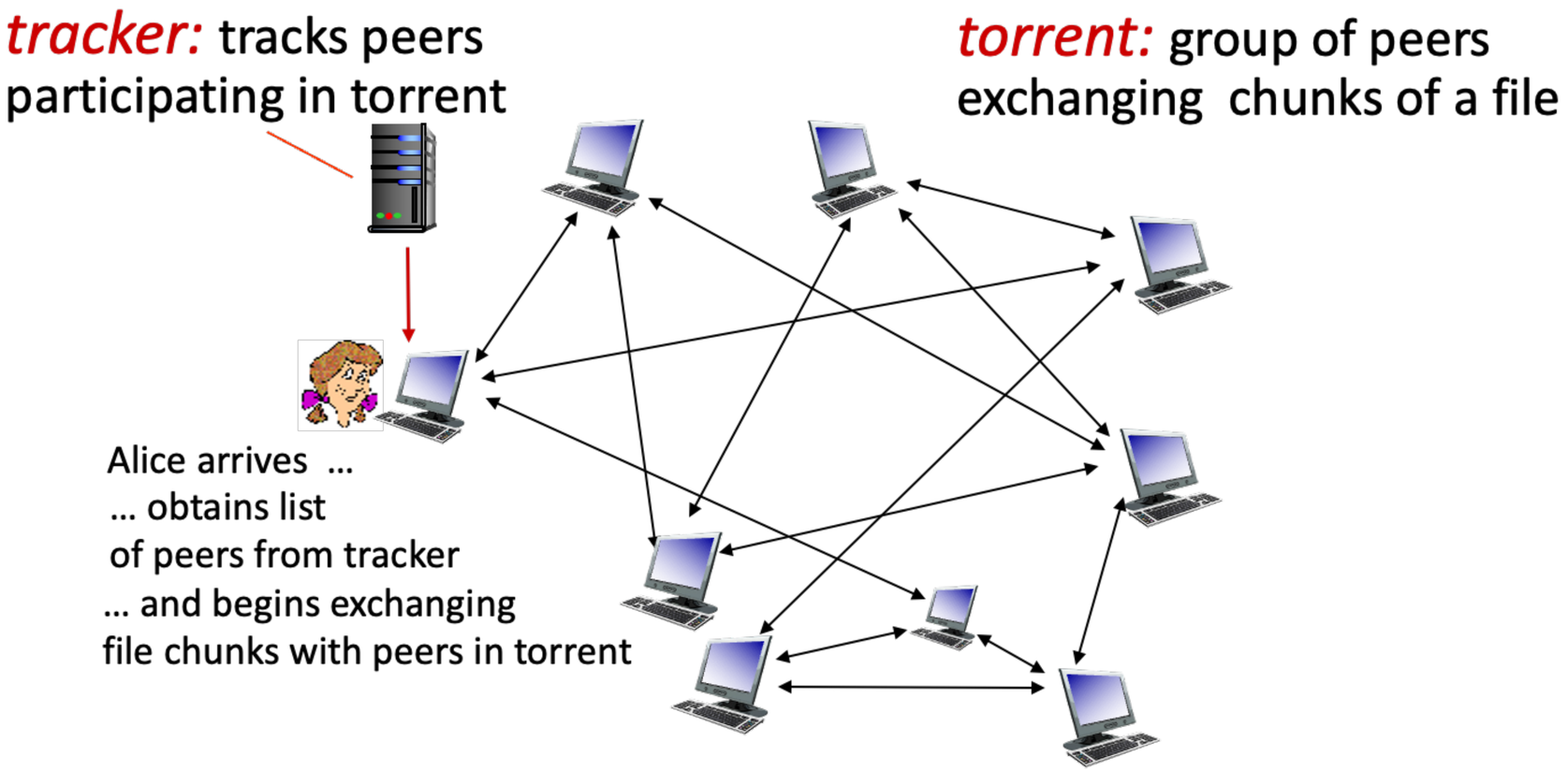
Client-server vs. P2P: example

- Suppose that
  - Client upload rate = u
  - $\frac{F}{u}$ = 1 hour
  - $u_s = 10u$
  - $d_{min} \geq u_s$



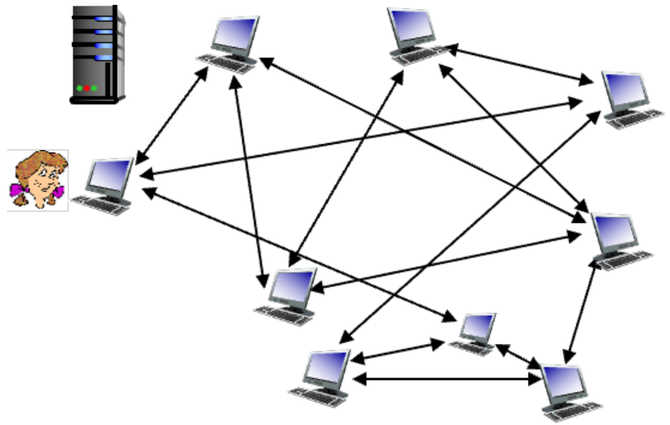
P2P file distribution: BitTorrent

- File divided into 256 Kb chunks
- Peers in torrent send/receive file chunks



P2P file distribution: BitTorrent

- Peer joining torrent
  - Has no chunks, but will accumulate them over time from other peers
  - Registers with tracker to get a list of peers, connects to subset of peers (“neighbors”)



- While downloading, peer uploads chunks to other peers
- Peer may change peers with whom it exchanges chunks
- churn: peers may come and go
- Once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent

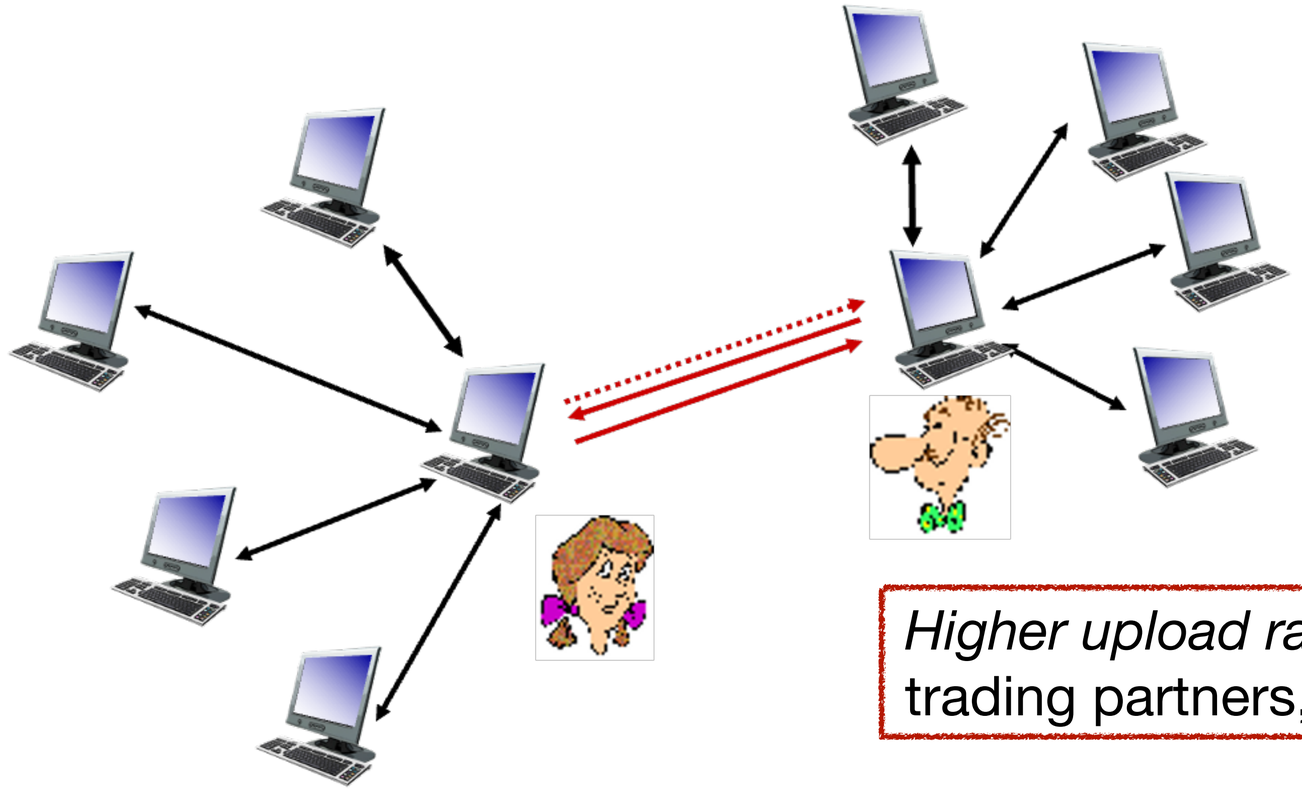
BitTorrent: requesting, sending file chunks

- Requesting chunks
  - At any given time, different peers have different subsets of file chunks
  - Periodically, Alice asks each peers for a list of chunks that they have
  - Alice requests missing chunks from peers, rarest first

- Sending chunks: tit-for-tat
  - Alice sends chunks to those four peers currently sending her chunks at highest rate
    - Other peers are choked by Alice (do not receive chunks from her)
    - Re-evaluate top 4 every 10 secs
  - Every 30 secs, randomly select another peer, starts sending chunks
    - “Optimistically unchoke” this peer
    - Newly chosen peer may join top 4

BitTorrent: tit-for-tat

- Alice “optimistically unchokes” Bob
- Alice becomes one of Bob’s top-four providers → Bob reciprocates
- Bob becomes one of Alice’s top-four providers



Higher upload rate: find better trading partners, get file faster!

Distributed Hash Table (DHT)

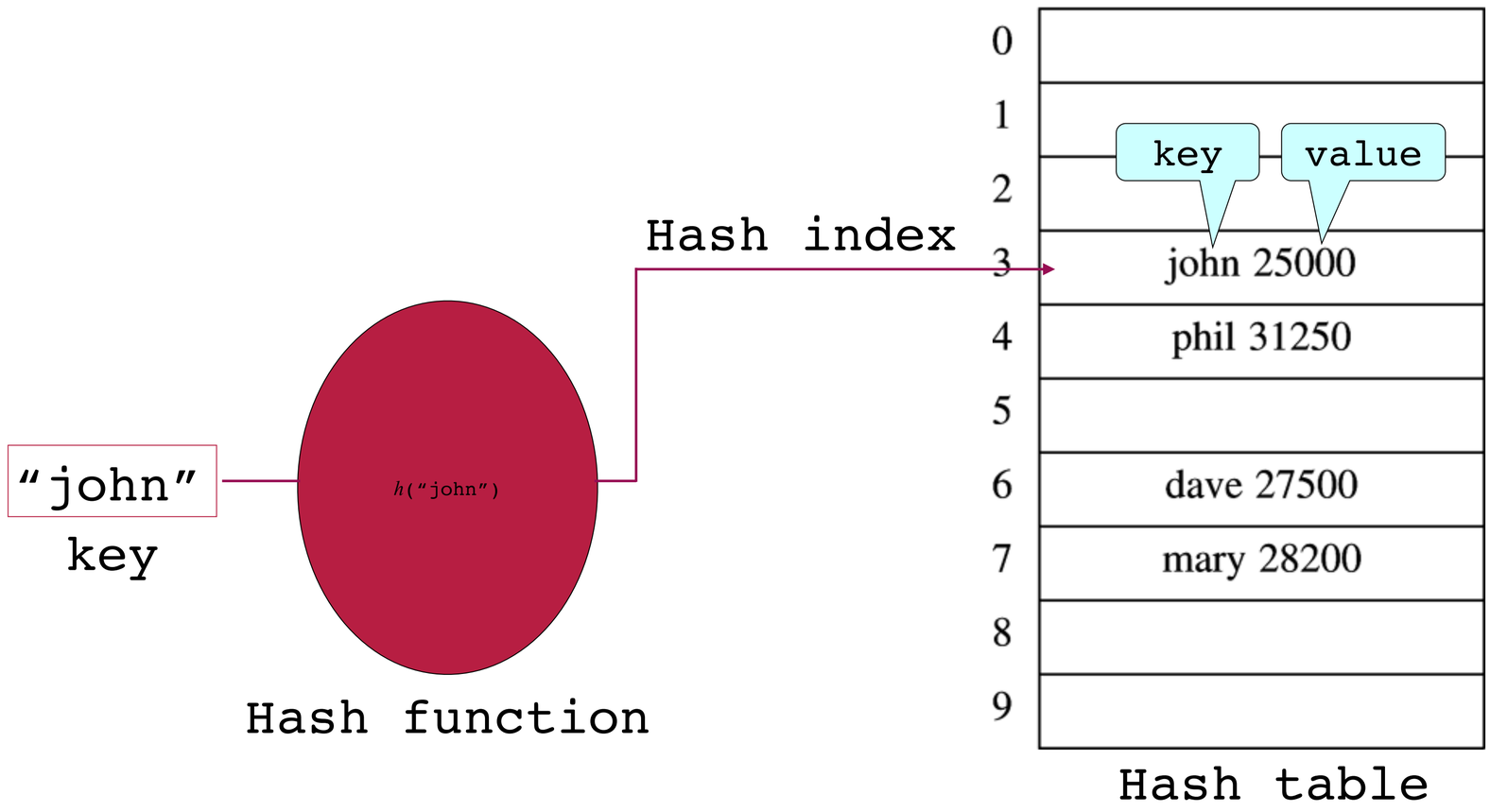
- DHT: a distributed P2P database
- Database has (key, value) pairs. e.g.,
  - key: social security number, value: human name
  - key: movie title, value: IP address
- Distributed the (key, value) pairs over the millions of peers
- A peer queries DHT with key
  - DHT returns values that match the key
- Peer can also insert (key, value) pairs

How to assign keys to peers?

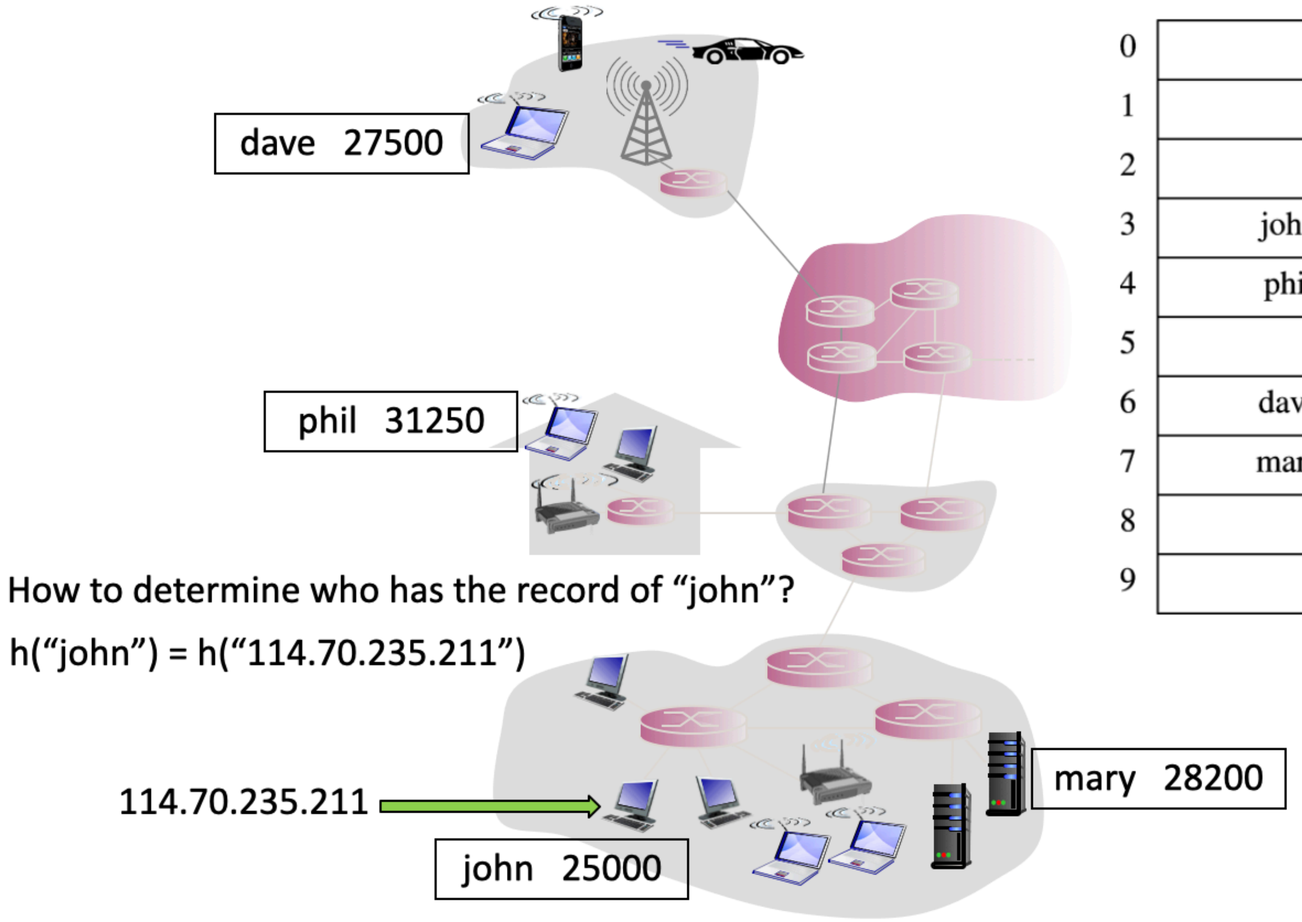
- Central issue
  - Assigning (key, value) pairs to peers

- Basic idea
  - Convert each key to an integer
  - Assign integer to each peer
  - Put (key, value) pair in the peer that is closest to the key

Hash table


0
1
2
3
4
5
6
7
8
9
key
value
john 25000
phil 31250
dave 27500
mary 28200
Hash index
"john"
key
h("john")
Hash function
Hash table

Distributed hash table



0	
1	
2	
3	john 25000
4	phil 31250
5	
6	dave 27500
7	mary 28200
8	
9	

DHT identifiers

- Assign integer identifier to each peer in range [0, 2^n-1] for some n
  - Each identifier is represented by n bits
- Require each key (hash index) to be an integer in the same range
- To get an integer key, hash original key
  - e.g., key = hash(“Led Zeppelin IV”)
  - This is why it is referred to as a distributed “hash” table

Assign keys to peers

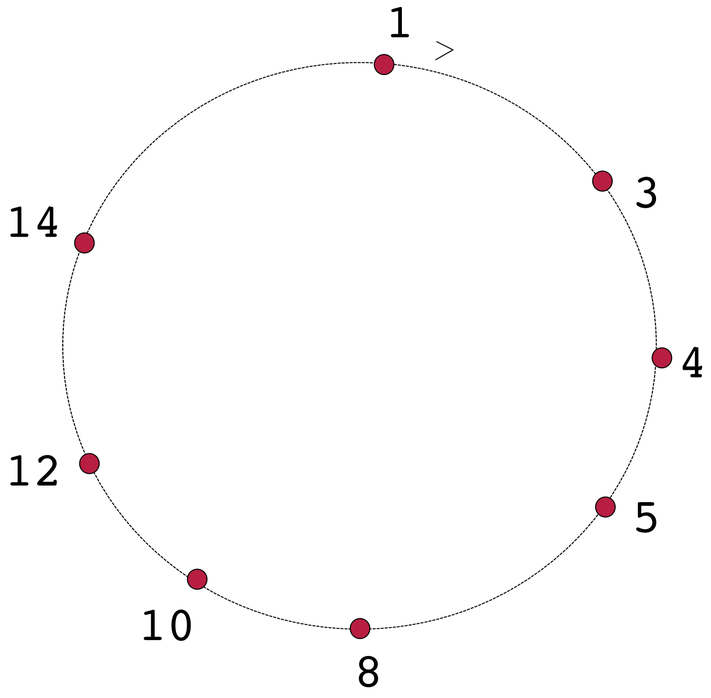
- Rule: assign key to the peer that has the closest ID

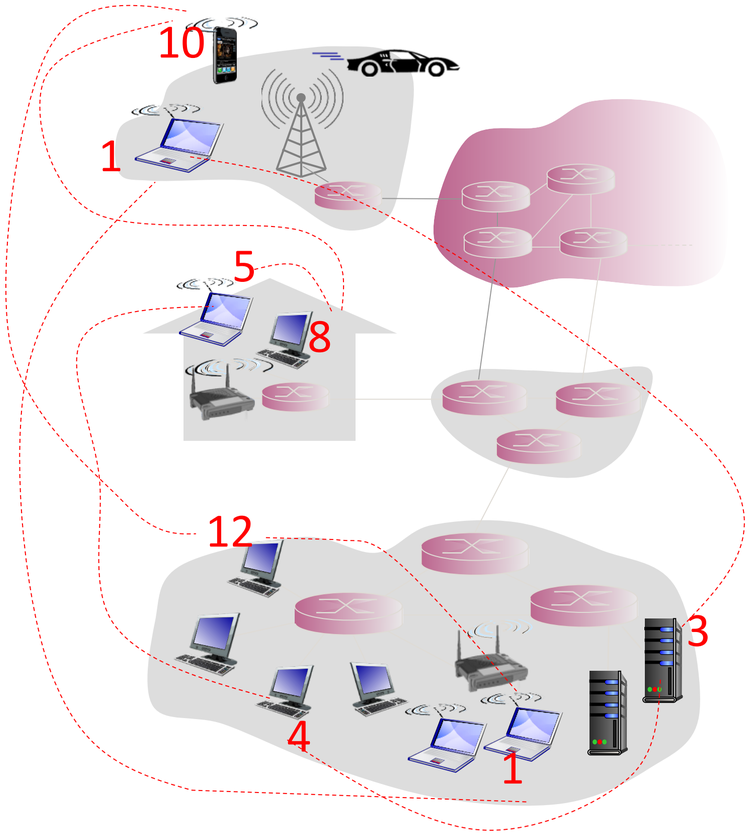
- Convention in lecture: closest is the immediate successor of the key

- e.g., $n = 4$ ($\rightarrow$ 16 peers can have unique IDs); peer IDs: 1, 3, 4, 5, 8, 10, 12, 14
  - key = 13, then successor peer's ID is 14
  - key = 15, then successor peer's ID is 1

Circular DHT

- Each peer is only aware of immediate successor and predecessor
- “Overlay network”



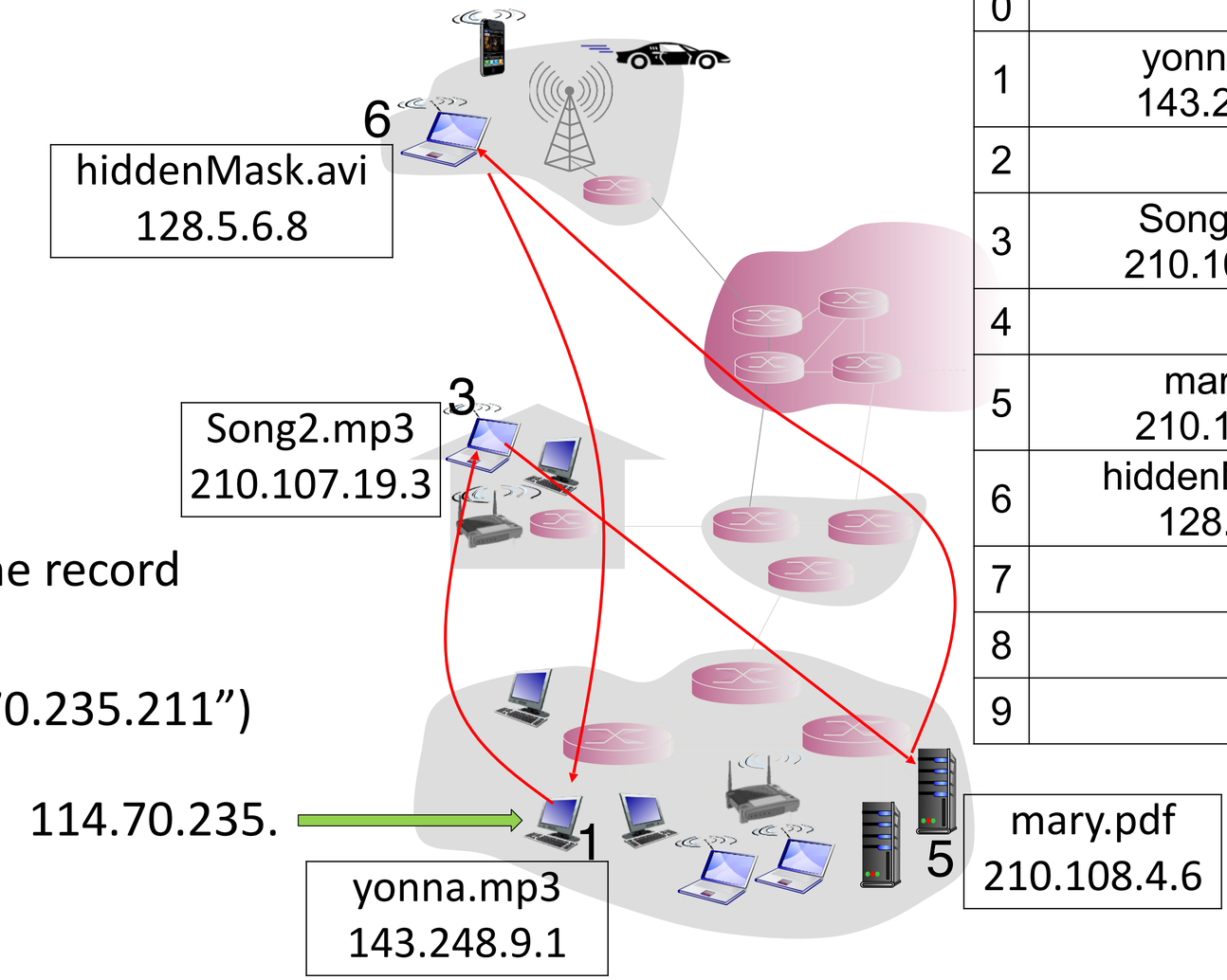


e.g. id= hash(IP address)

Distributed hash table - file sharing example

How to determine who has the record of “yonna.mp3”?

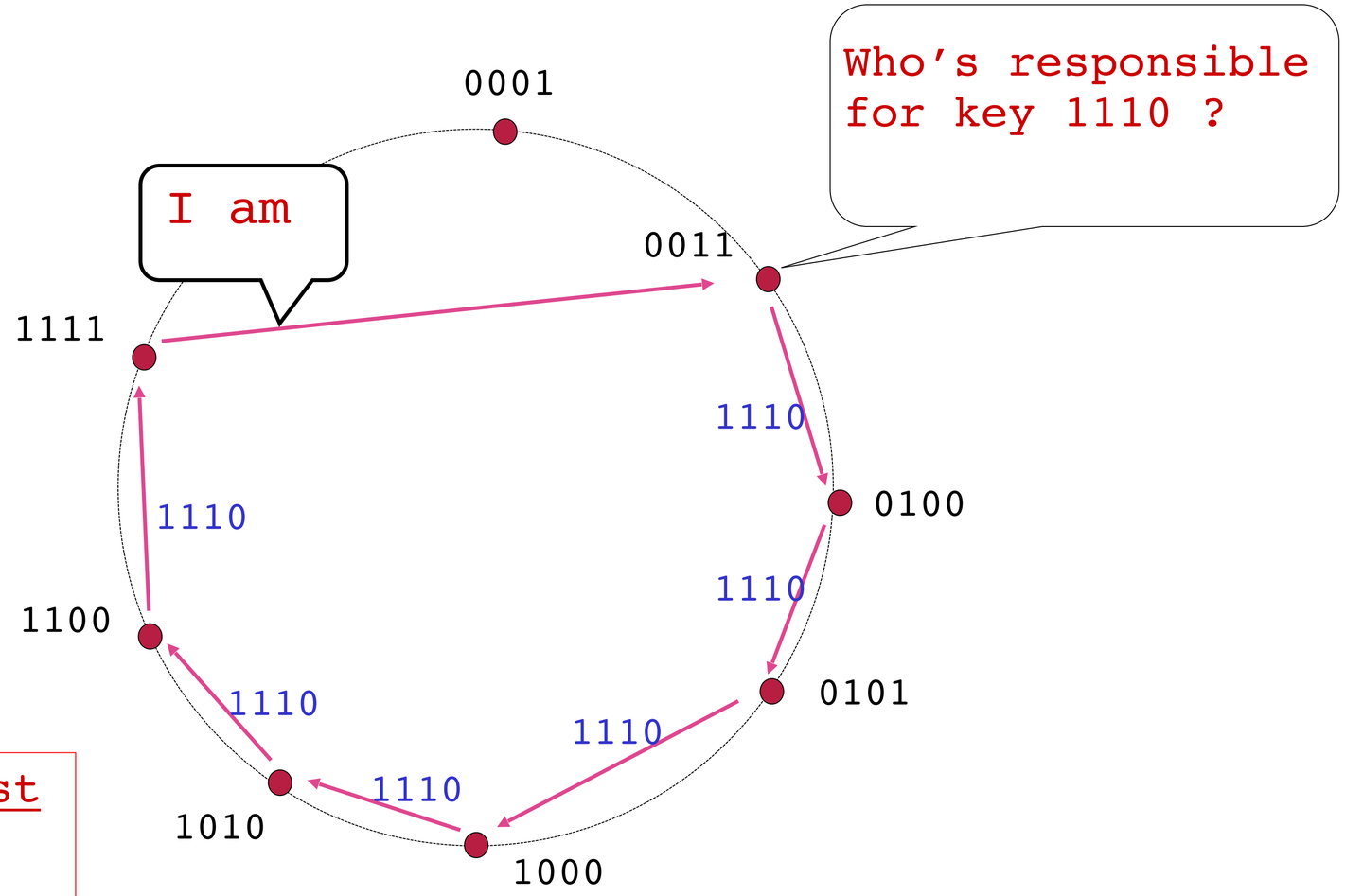
⇒ h(“yonna.mp3”) = h(“114.70.235.211”)



0	
1	yonna.mp3 143.248.9.1
2	
3	Song2.mp3 210.107.19.3
4	
5	mary.pdf 210.108.4.6
6	hiddenMask.avi 128.5.6.8
7	
8	
9	

Circular DHT

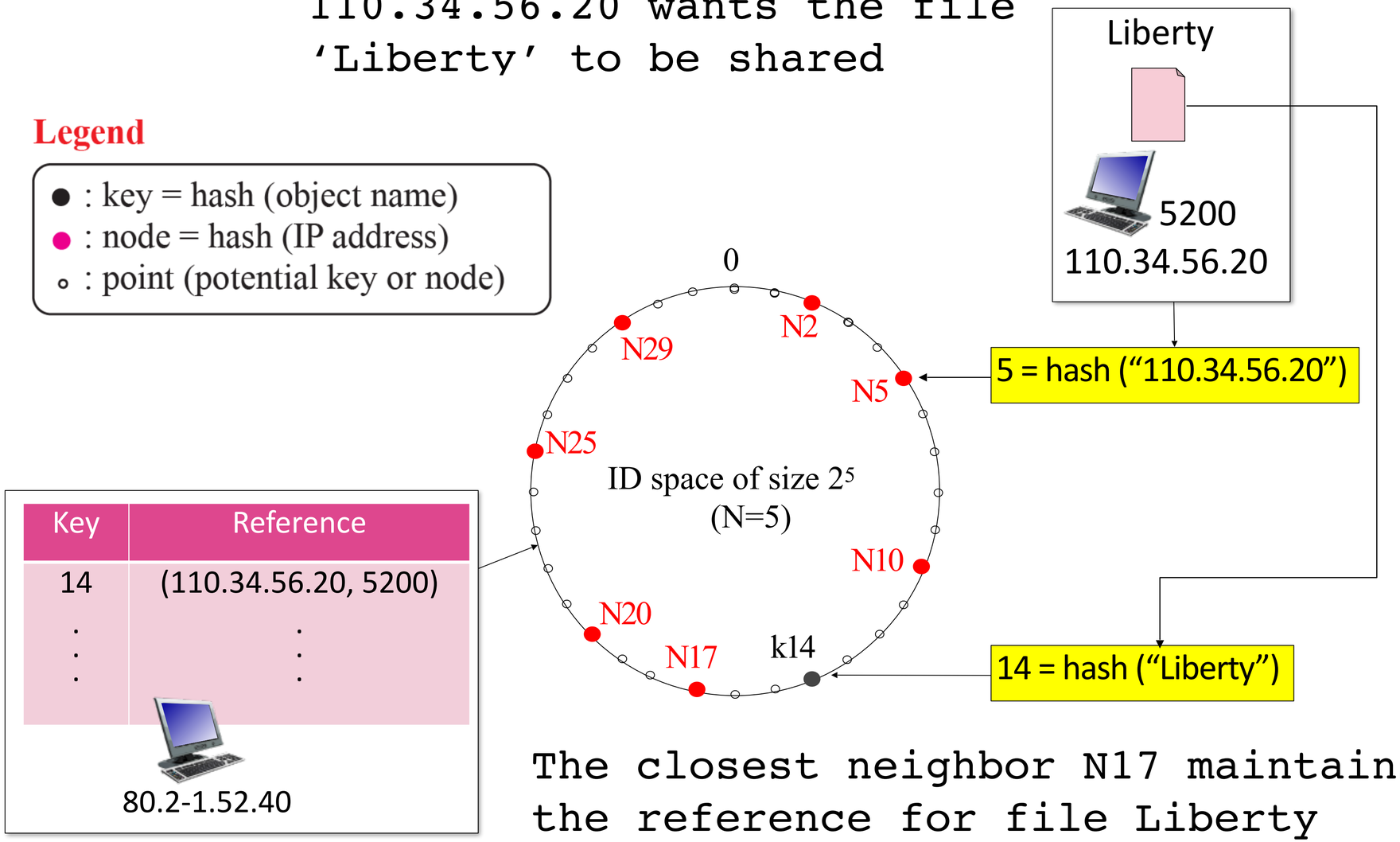
- O(*N*) messages on average to resolve query, when there are *N* peers



Define closest as closest successor

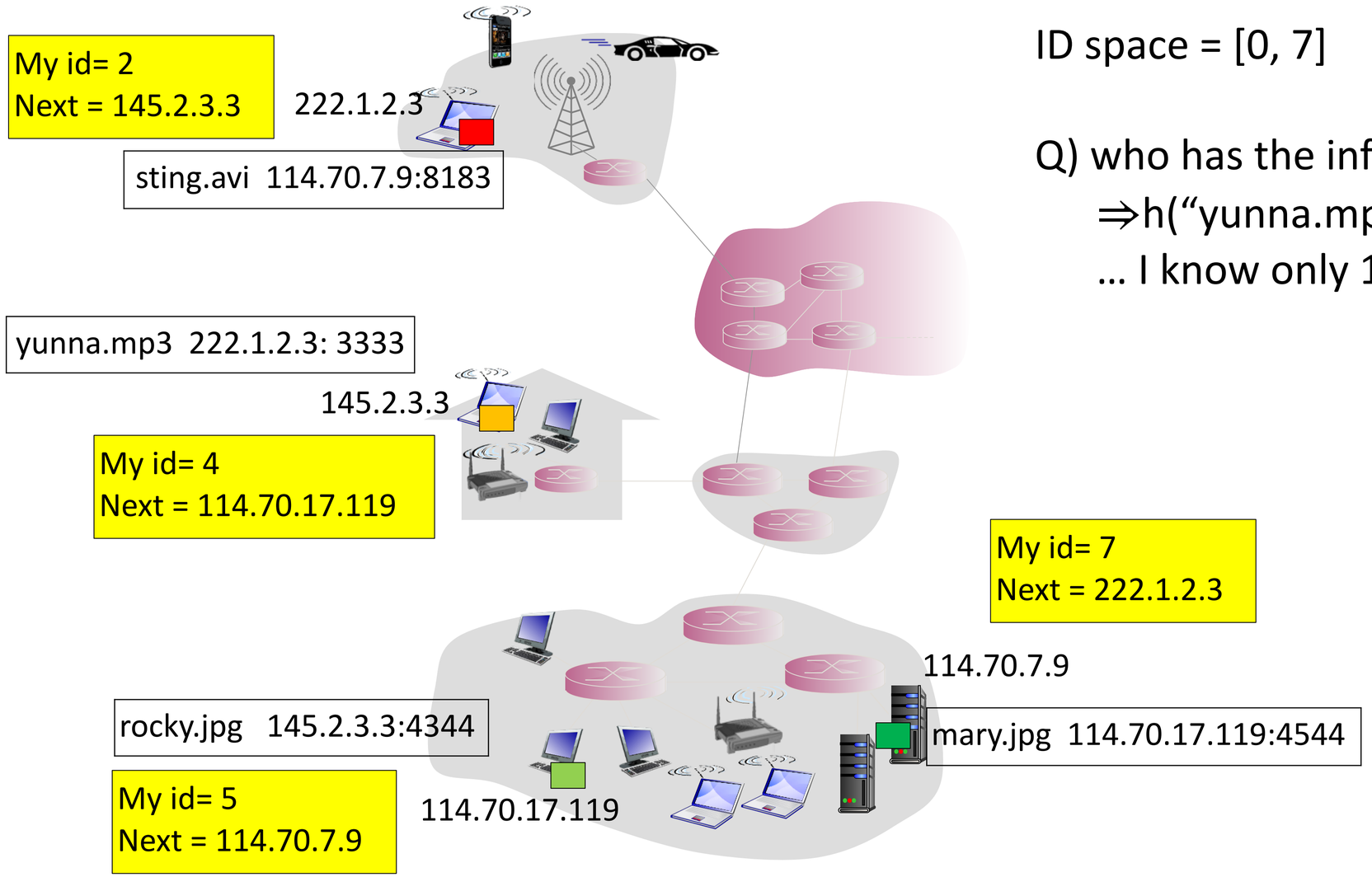
Distributed hash table - file sharing example

110.34.56.20 wants the file 'Liberty' to be shared



The closest neighbor N17 maintains the reference for file Liberty

Distributed hash table - distributed file system

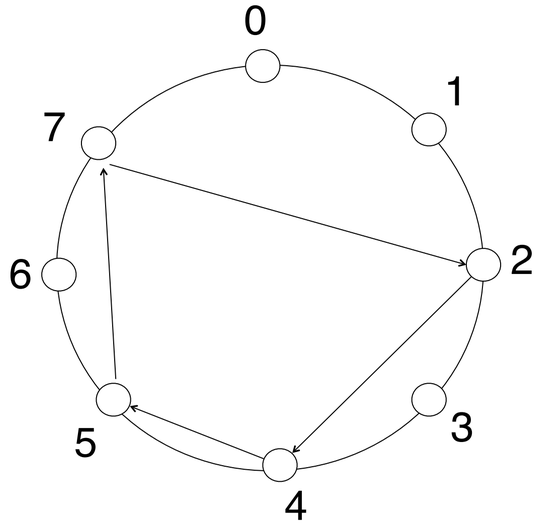


ID space = [0, 7]

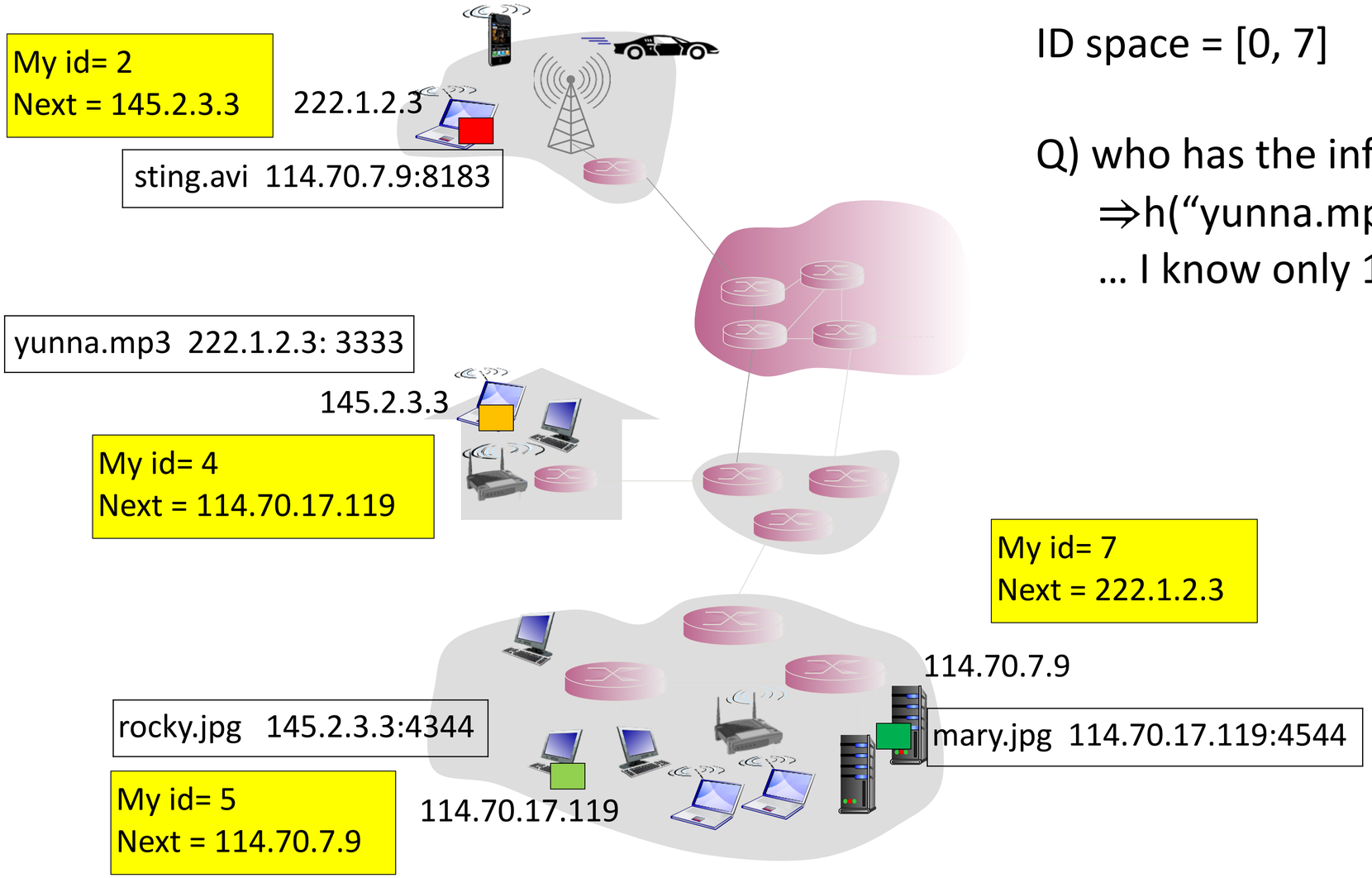
Q) who has the information about “yunna.mp3”?

⇒h(“yunna.mp3”) = h(“145.2.3.3”) = 4

… I know only 114.70.7.9



Distributed hash table - distributed file system

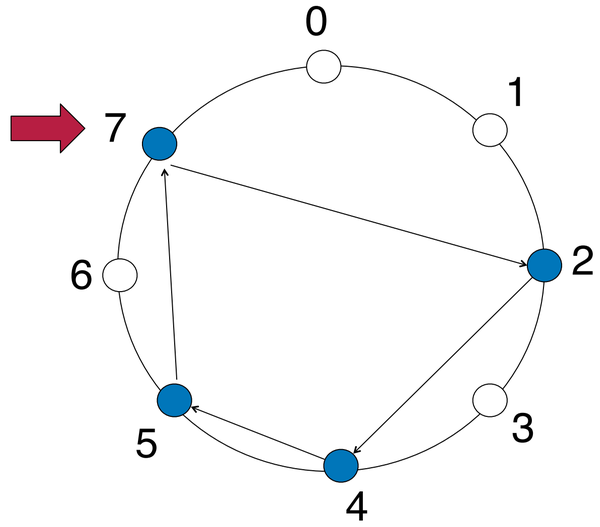


ID space = [0, 7]

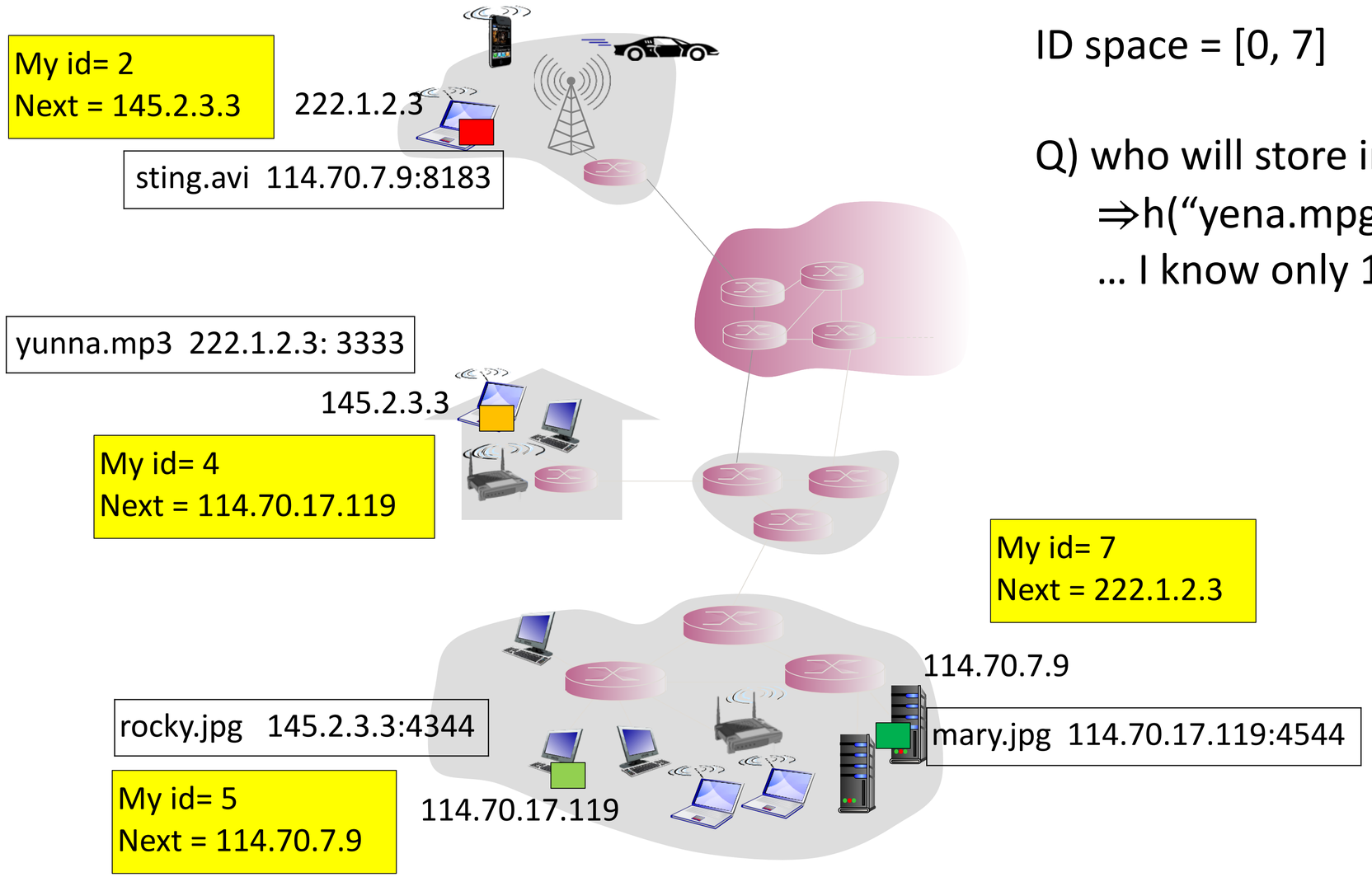
Q) who has the information about “yunna.mp3”?

⇒h(“yunna.mp3”) = h(“145.2.3.3”) = 4

… I know only 114.70.7.9



Distributed hash table - distributed file system

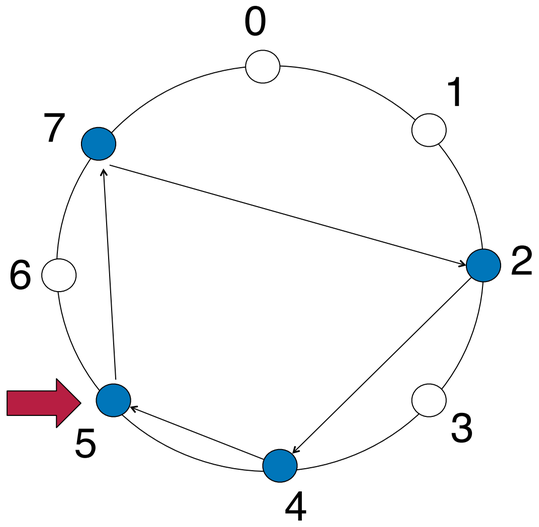


ID space = [0, 7]

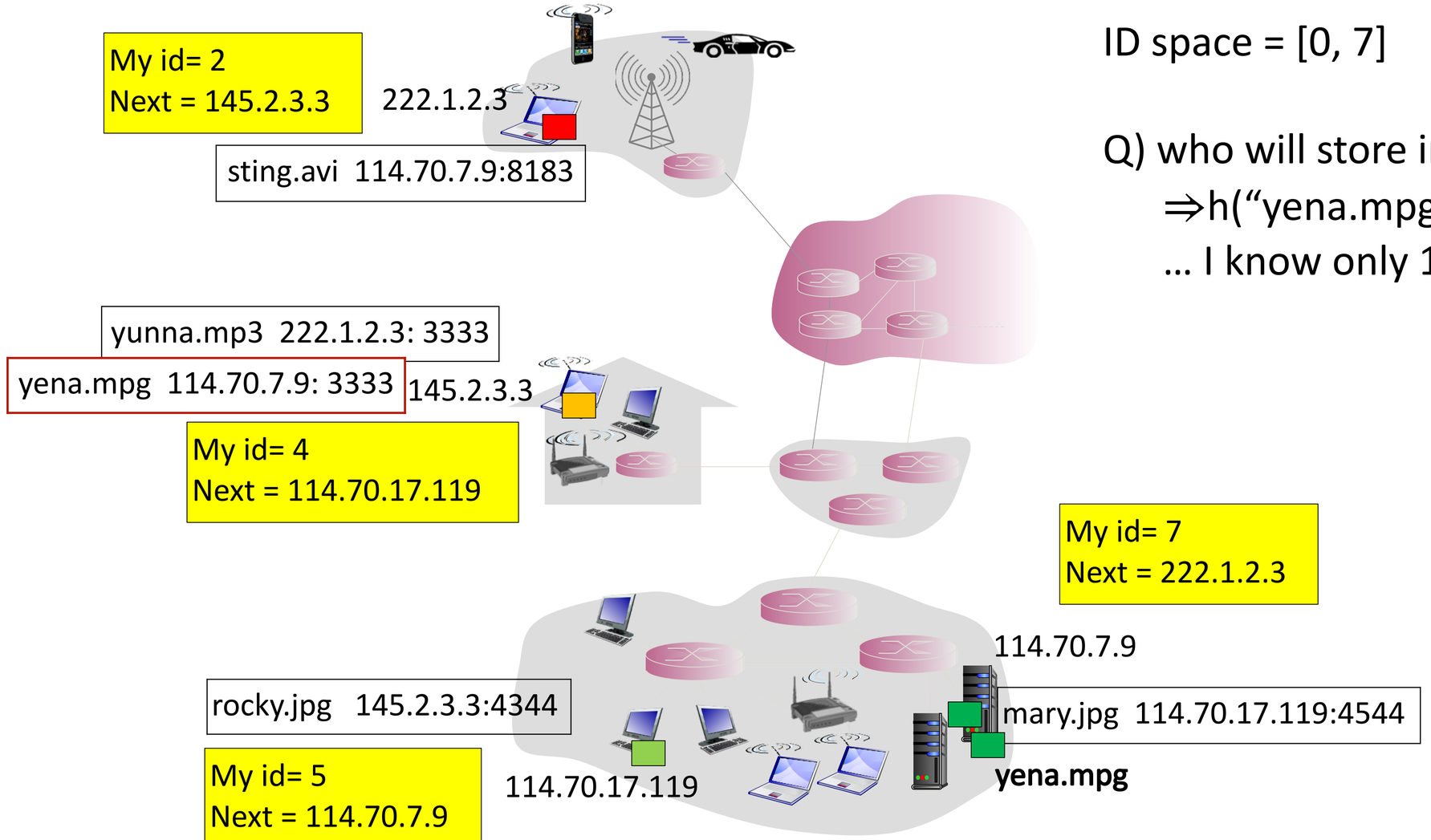
Q) who will store info of “yena.mpg”?

⇒h(“yena.mpg”) = 3

… I know only 114.70.17.119



Distributed hash table - distributed file system

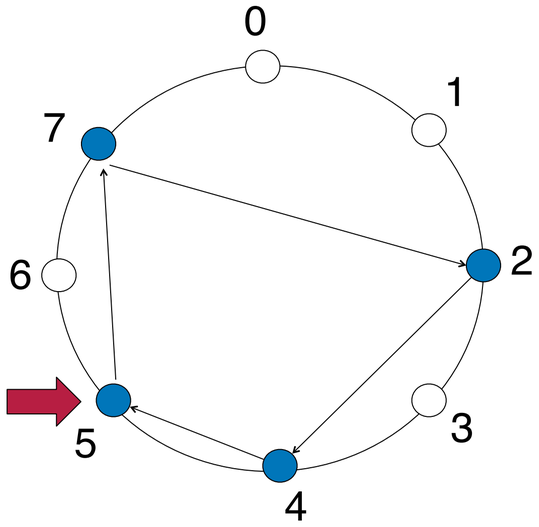


ID space = [0, 7]

Q) who will store info of “yena.mpg”?

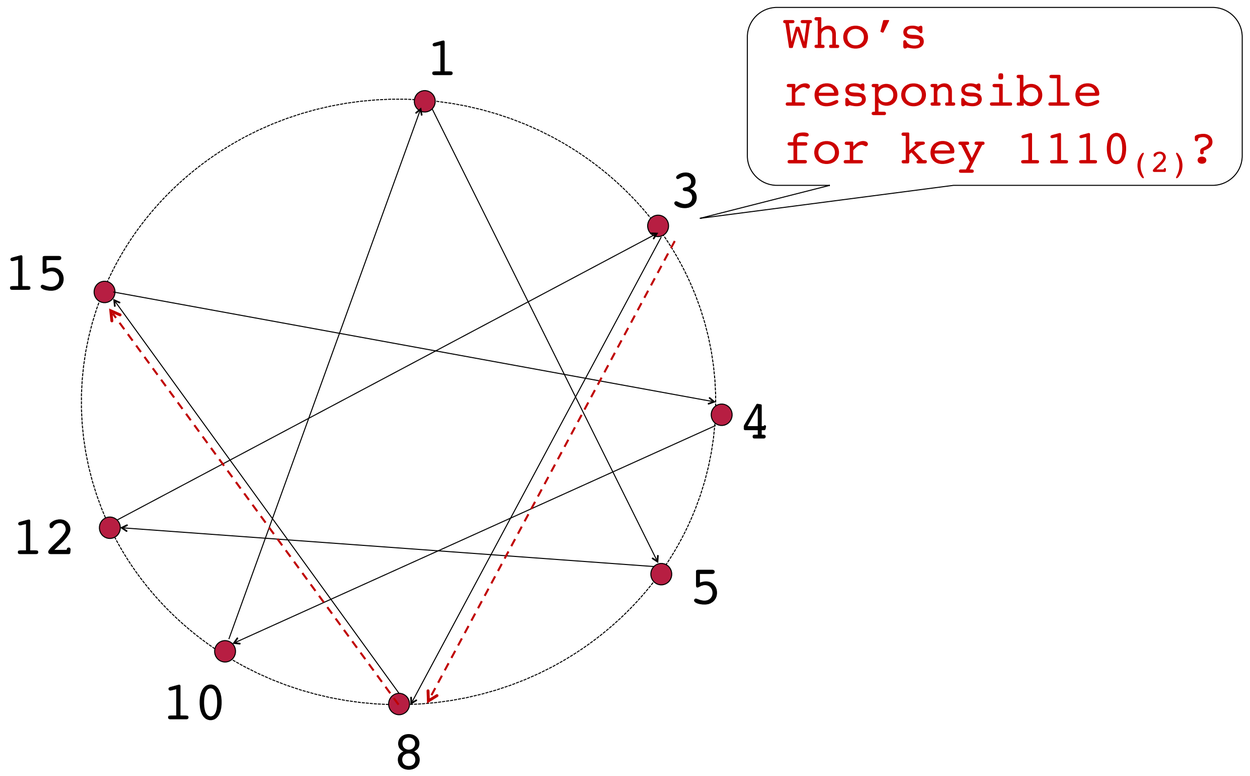
⇒h(“yena.mpg”) = 3

… I know only 114.70.17.119

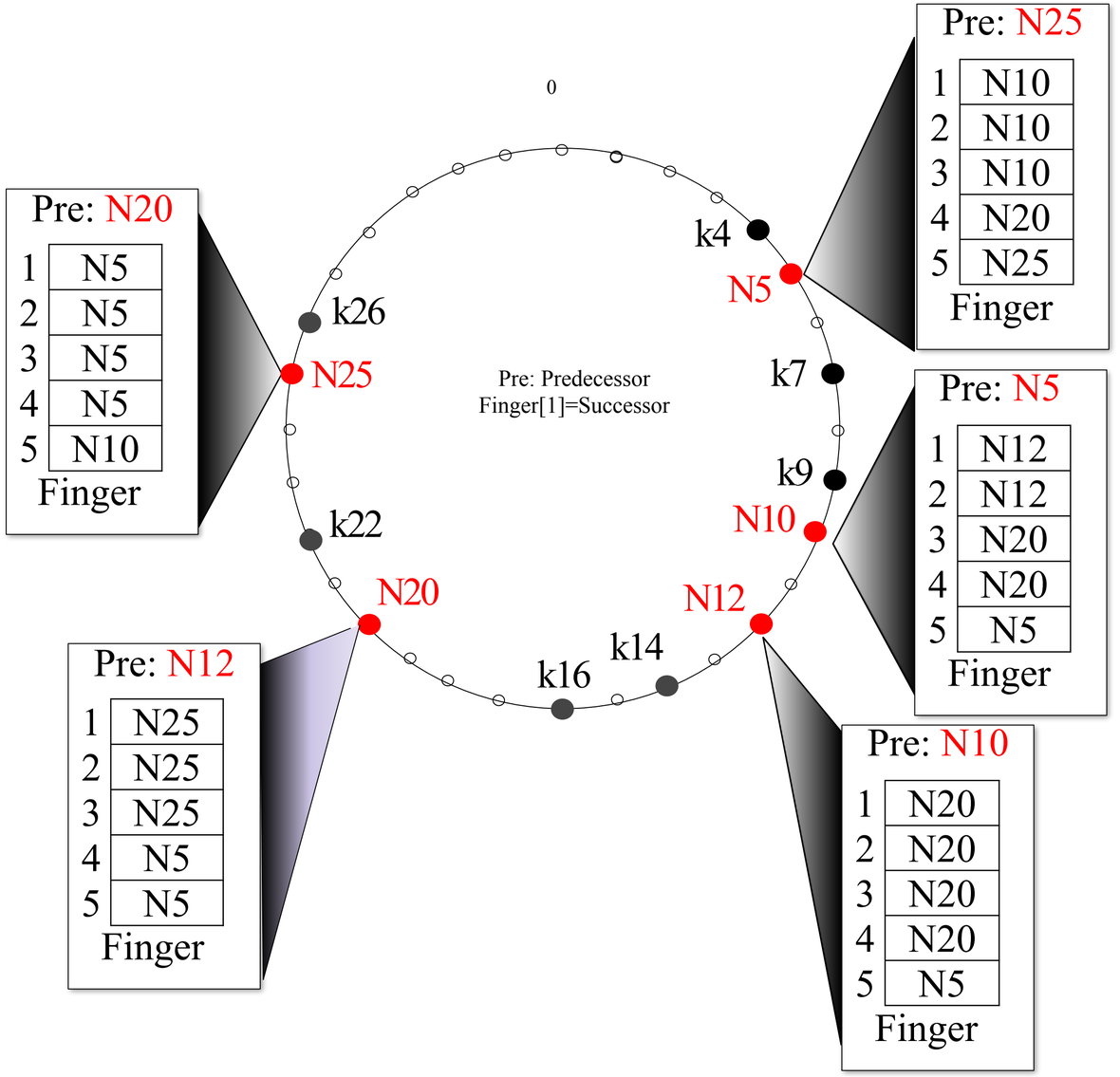


Circular DHT with shortcuts

- Each peer keeps track of IP addresses of predecessor, successor, shortcuts.
- Message transmission is reduced from 6 to 2
- Possible to design shortcuts so O(log *N*) neighbors, O(log *N*) message transmission in query



Example of predecessor-successor list



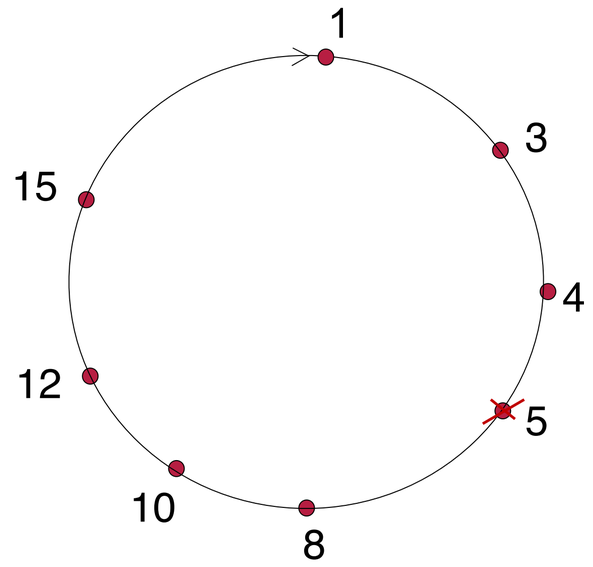
Legend

- ● : key = hash (object name)
- ● : node = hash (IP address)
- ∘ : point (potential key or node)

Peer churn

- Handling peer churn
  - Peers may come and go (churn)
  - Each peer knows address of its two successors
  - Each peer periodically pings its two successors to check aliveness
  - If immediate successor leaves, choose next successor as new immediate successor

- e.g., peer 5 abruptly leaves
  - Peer 4 detects peer 5 departure → makes 8 its immediate successor
  - Asks 8 who its immediate successor is
  - Makes 8's immediate successor its second successor
  - What if peer 13 want to join?



Questions?